

Dossier de contexto para documentación

Mercado Secundario IFC sobre Avalanche — Hackathon Avalanche LATAM 2026

*Este documento es la **fuentes** que la persona que documente usará para responder, ítem por ítem, las seis preguntas del brief: **qué es, por qué se eligió, para qué sirve, estado, dónde vive en el repo, y quién lo necesita entender**. Cada bloque ya viene escrito en el orden que esas seis respuestas requieren — sólo hay que reformular en el documento final.*

0. La idea del proyecto

Qué construimos en una frase

Una plataforma white-label que cualquier IFC mexicana puede adoptar para emitir participaciones tokenizadas y operar un mercado secundario regulado entre inversionistas calificados, con cumplimiento CNBV embebido a nivel de smart contract — sobre Avalanche.

El problema, en concreto

México tiene un mercado de equity crowdfunding regulado por CNBV (Ley Fintech, 2018). Hoy operan plataformas como Arkangeles, Play Business, Snowball, Bankaool y otras Instituciones de Financiamiento Colectivo (IFC). Estas plataformas conectan startups y PyMEs que necesitan capital con inversionistas individuales que aportan tickets pequeños.

El problema es que el inversionista, una vez que entra a una oferta, **queda atrapado entre 5 y 10 años**. Su única vía de salida es esperar a que la empresa sea adquirida, salga a bolsa, o pague dividendos. No puede vender su participación a otro inversionista porque no existe un mercado secundario regulado para participaciones IFC. Esto:

- **Deprime la demanda primaria:** muchos inversionistas que estarían dispuestos a entrar no lo hacen al ver que no podrán salir.
- **Limita el ticket promedio:** el inversionista compromete menos dinero porque sabe que estará bloqueado.
- **Aleja al inversionista institucional:** fondos requieren liquidez y reportes regulares.
- **Genera un cap table operativamente costoso:** la IFC debe reconciliar titularidad en Excel entre fundadores, plataforma e inversionistas, con riesgo legal por errores.
- **Repite KYC:** cada plataforma hace verificación de identidad desde cero, frustrando al usuario.
- **Audit log no es inherentemente inmutable:** las IFCs guardan registros propios, pero CNBV exige un rastro auditable que demuestre integridad, lo cual cuesta operativamente.

Por qué esto es atacable ahora

Tres tendencias convergen:

1. **Tokenización de RWA (Real-World Assets)** está madurando en infraestructura blockchain. Estándares como ERC-3643 / T-REX permiten security tokens regulados con identidad on-chain.
2. **Avalanche Subnets** permiten correr una blockchain custom con validadores propios, gas pagable en stablecoin, y reglas permissioned a nivel de protocolo — exactamente lo que un regulador como CNBV puede aceptar.
3. **CNBV está abierta a sandbox:** la Ley Fintech contempla la figura del "modelo novedoso" para aprobar arquitecturas no convencionales.

Cómo lo resolvemos — la arquitectura del producto

Construimos cuatro capas desacopladas:

1. **Capa on-chain (Avalanche):** smart contracts ERC-3643 con identity registry separada del token, y módulos de compliance intercambiables (lockup, max holders, jurisdicción, accreditation). Cada operación de transfer pasa por estos módulos antes de ejecutarse — el cumplimiento no es opcional, vive en el código.
2. **Capa de datos (Postgres + Redis + IPFS):** indexer que mantiene una copia queryable del estado on-chain (cap_table, orders, trades, audit_log). IPFS para prospectos y documentos legales.
3. **Capa backend (Node.js / Next.js API):** KYC orchestrator, matching engine off-chain (firmas EIP-712), notifications, RBAC, audit log immutable.
4. **Capa frontend (Next.js + React):** tres portales — uno para inversionistas (descubrir ofertas, comprar/vender, ver portfolio), uno para emisores IFC (crear ofertas, ver cap table, distribuir dividendos), uno para compliance (KYC, freeze, audit).

Por qué es defensible (el moat)

- **Compliance embebido:** un DEX genérico no puede operar como mercado secundario regulado en México porque no enforza jurisdicción ni accreditation a nivel protocolo. Nosotros sí.
- **Subnet propia:** validadores controlados por la IFC + auditor → CNBV puede inspeccionar la red, cosa imposible en una public chain.
- **KYC reusable:** identidad firmada por un ClaimIssuer es portable entre plataformas que reconozcan al issuer — efecto de red institucional.
- **Audit log doble:** tabla append-only en Postgres + eventos on-chain inmutables. Cualquiera de los dos sirve a CNBV; tener los dos es defense in depth.

Modelo de negocio

- Fee de **emisión** (one-time por oferta deployada): paga la IFC.
- Fee de **trading** (basis points sobre el notional): paga el trader.
- **Custodia + liquidación** (suscripción mensual): paga la IFC.

- **White-label a otras IFCs:** licencia anual + setup. Cada IFC opera su propia instancia con sus colores y dominio.

Cliente piloto y mercado expandible

- **Piloto:** Arkangeles (IFC mexicana de equity crowdfunding regulada por CNBV).
- **Expandible inmediato:** Bankaool, Play Business, Snowball, otras IFCs MX.
- **LATAM:** Brasil (CVM), Colombia (Superfinanciera), Chile (CMF) tienen marcos similares.

Lo que NO somos

- No somos un DEX (no permitimos trading anónimo).
- No somos una bolsa de valores (no operamos bajo Ley del Mercado de Valores; participaciones IFC son una categoría distinta bajo Ley Fintech).
- No somos una wallet (consumimos wallets existentes vía wagmi/RainbowKit).
- No somos un KYC provider (consumimos uno; hoy mock, mañana Truora/Mati).

Estado actual

MVP funcional construido en hackathon. Backend real con auth + DB + 22 endpoints API. Frontend completo con tres portales (~30 páginas). Mock blockchain layer que se comporta como la cadena real con la misma interfaz SDK que tendrá el adapter Avalanche real (swap = cambiar `mode: 'mock'` por `mode: 'avalanche'`). Smart contracts escritos como esqueleto, pendientes de tests, auditoría y deploy. Wallet integration (Core Wallet de Avalanche, MetaMask, Coinbase, Rabby) operativa. Light/dark mode con tokens semánticos. Hero animado en marca Arkangeles (azul electric).

Audiencias y qué busca cada una

- **Jurado del hackathon** (técnico + business): evalúa viabilidad real, calidad del código, uso inteligente de Avalanche, modelo de negocio. Quiere ver demo en vivo + repo limpio + pitch coherente.
 - **Equipo (futuro nosotros):** necesita levantar el proyecto en una máquina nueva en menos de 30 minutos.
 - **Arkangeles y otras IFCs:** clientes potenciales. Evalúan si esto reduce su costo operativo y si CNBV lo aceptaría. Quieren ver compliance + modelo de adopción.
 - **Auditor de smart contracts:** revisará seguridad pre-deploy. Quiere docs de threat model + tests + invariantes.
 - **CNBV / equipo legal de la IFC:** valida cumplimiento regulatorio. Quiere mapeo artículo-por-artículo.
-

1. Problemas que resolvemos — contexto extendido

Ilíquidez del inversionista IFC

Qué es: tras invertir en una oferta IFC, el inversionista no puede vender hasta una salida del emisor (5–10 años en promedio). **Por qué importa:** deprime la demanda primaria; el inversionista promedio reporta esto como su barrera #1. **Cómo resolvemos:** mercado secundario regulado entre inversionistas previamente acreditados, settlement atómico contra stablecoin. **Estado:** UI funcional, contratos pendientes de deploy. **Vive en:** `apps/web/src/app/(app)/investor/offerings/[id]/orderbook` , `packages/contracts/contracts/market/Settlement.sol` .

Cap table manual en Excel

Qué es: las IFCs reconcilian titularidad accionaria entre fundadores, plataforma e inversionistas usando hojas de cálculo que se desincronizan. **Por qué importa:** errores de cap table tienen impacto legal directo. **Cómo resolvemos:** cap table es la blockchain misma, mantenida en `cap_table` (Postgres) por el indexer que escucha eventos `Transfer` . **Estado:** estructura lista (Prisma + indexer), datos reales requieren contratos deployados. **Vive en:** `packages/database/prisma/schema.prisma` (CapTableEntry), `apps/indexer/src/handlers/transfer.handler.ts` .

KYC repetido por plataforma

Qué es: el inversionista hace KYC desde cero en cada IFC nueva. **Por qué importa:** fricción que hace abandonar onboarding. **Cómo resolvemos:** `IdentityRegistry` on-chain con claims firmadas por un `ClaimIssuer` reusable; otras IFCs pueden reconocer la misma identidad. **Estado:** arquitectura definida; en este hackathon el KYC mismo es mock (UI sin provider real). **Vive en:** `packages/contracts/contracts/identity/` , `apps/web/src/app/api/kyc/` .

Distribución de dividendos manual

Qué es: distribuir dividendos a 200+ holders requiere transferencias bancarias manuales. **Por qué importa:** costo operativo + riesgo de error. **Cómo resolvemos:** smart contract (planeado, no implementado este hackathon) que hace pro-rata airdrop al settlement de dividendos. **Estado:** pendiente.

Compliance no auditable

Qué es: CNBV exige registro inmutable de operaciones; las IFCs lo guardan en logs propios que no son inherentemente inmutables. **Por qué importa:** Disposiciones de Carácter General lo exigen. **Cómo resolvemos:** tabla `audit_log` append-only en Postgres + eventos on-chain (Transfer, ForcedTransfer, WalletFrozen) que son inmutables por diseño. **Estado:** `audit_log` funcional, eventos on-chain pendientes de contratos reales. **Vive en:** `apps/web/src/lib/server/services/audit.service.ts` , modelo `AuditLog` en Prisma.

Acceso limitado al mercado secundario de equity privado

Qué es: en México no hay un mercado secundario líquido para participaciones en startups privadas. **Por qué importa:** secundario es lo que da múltiplo a la primaria. **Cómo resolvemos:** orderbook con matching engine off-chain + settlement on-chain. **Estado:** UI animada y matching mock; settlement pendiente.

Onboarding lento de inversionistas

Qué es: KYC + verificación + acreditación tardan días. **Por qué importa:** cada día de onboarding pierde inversionistas. **Cómo resolvemos:** flujo de 4 pasos con estados visibles; reuso de identidad para futuras ofertas. **Estado:** flujo UI completo, KYC backend mock. **Vive en:** `apps/web/src/app/(onboarding)/`.

Falta de transparencia en titularidad

Qué es: el inversionista no puede verificar de forma independiente cuántos tokens posee y quién más posee. **Por qué importa:** confianza institucional. **Cómo resolvemos:** cap table on-chain pública (vía explorer de la subnet), wallet del inversionista lo verifica directo. **Estado:** depende de contratos deployados.

Costos altos de cumplimiento CNBV

Qué es: mantener registros conforme a Disposiciones tiene costo operativo significativo. **Por qué importa:** limita el margen de las IFCs. **Cómo resolvemos:** compliance modular, audit log automático. **Estado:** arquitectura validada en mock; valor real depende de adopción.

Imposibilidad de forced transfer en infra tradicional

Qué es: si un inversionista pierde llaves o muere, recuperar las participaciones requiere proceso legal lento. **Por qué importa:** requisito CNBV implícito (recovery). **Cómo resolvemos:** `forcedTransfer` y `freezeWallet` en `SecurityToken.sol`, ejecutable por compliance officer (con audit log). **Estado:** implementado en código Solidity, pendiente deploy.

2. Qué construimos — contexto por entregable

Plataforma web institucional con tres portales

Investor, Issuer y Compliance Admin. Cada portal tiene navegación, layouts, y vistas propias. **Por qué importa:** cada rol opera con diferentes permisos y necesita diferentes vistas. **Vive en:** `apps/web/src/app/(app)/{investor,issuer,admin}/`. **Estado:** ~30 páginas funcionales.

Landing institucional

Hero con shader cosmic animado, métricas, "cómo funciona", arquitectura como timeline orbital interactivo, FAQ, footer. **Por qué importa:** convierte visitantes a registrados. **Vive en:**

`apps/web/src/app/page.tsx` y `apps/web/src/components/landing/`. **Estado:** completo, light/dark mode soportado.

Mercado secundario con orderbook

Place order (firmado EIP-712), matching engine, settlement atómico. **Por qué importa:** es el corazón del producto. **Vive en:** `apps/web/src/lib/server/services/order.service.ts` + `matching.service.ts`, contratos en `packages/contracts/contracts/market/`. **Estado:** matching mock, settlement pendiente.

Sistema de identidad on-chain (ERC-3643)

`IdentityRegistry` mapea wallet a identidad legal verificada con claims firmadas. **Por qué importa:** base del cumplimiento — sin identidad no se puede operar. **Vive en:** `packages/contracts/contracts/identity/IdentityRegistry.sol`. **Estado:** contrato escrito, no deployado.

KYC orchestrator

Endpoint para iniciar KYC + webhook idempotente que recibe el resultado del provider, firma claim como ClaimIssuer y registra identidad on-chain. **Por qué importa:** bridge entre KYC tradicional (Truora/Mati) e identidad blockchain. **Vive en:** `apps/web/src/app/api/kyc/`. **Estado:** backend funcional, provider mock.

Indexer event-driven

Worker Node.js que consume Redis Streams (eventos del mock chain), aplica handlers tipados (Transfer → CapTable, TradeExecuted → Trade, etc.) con idempotencia. **Por qué importa:** mantiene Postgres sincronizado con on-chain sin lógica duplicada. **Vive en:** `apps/indexer/`. **Estado:** funcional con mock chain.

API REST con auth real, RBAC y audit log

22 endpoints, validación Zod, error mapper, rate limiting Redis sliding window. **Por qué importa:** backend del producto. **Vive en:** `apps/web/src/app/api/`. **Estado:** producción-ready (excepto los que dependen de chain real).

Mock blockchain SDK

`@hack/sdk` define interfaces (IdentityRegistryAdapter, etc.) e implementaciones in-memory que emiten eventos al EventBus → Redis Stream → indexer. La interfaz es la misma que tendrá el adapter Avalanche real. **Por qué importa:** permite construir todo el frontend/backend sin esperar contratos deployados; swap mock→real es cambiar `mode: 'mock'` por `mode: 'avalanche'`. **Vive en:** `packages/sdk/src/blockchain/`. **Estado:** mock completo, real adapter pendiente.

Smart contracts (esqueleto)

13 contratos Solidity que cubren identidad, compliance modular, security token, factory, market, settlement, escrow. **Por qué importa:** son lo que hace que el sistema sea *blockchain-native*, no sólo una BD distribuida. **Vive en:** `packages/contracts/contracts/`. **Estado:** esqueleto sin tests ni deploy.

Wallet integration con Core Wallet priorizada

Modal RainbowKit con Core Wallet (Avalanche-native) en grupo "Avalanche", MetaMask/Coinbase/Rabby/Rainbow/Trust en "Más opciones", WalletConnect opcional. **Por qué importa:** la barrera de adopción es la conexión de wallet — Core es la oficial de Avalanche. **Vive en:** `apps/web/src/lib/client/wagmi.ts` , `apps/web/src/providers/Web3Provider.tsx` . **Estado:** producción-ready.

Dashboard con balances reales AVAX/USDC

Lectura en vivo del RPC de Fuji vía wagmi `useBalance` . **Por qué importa:** prueba que la wallet del usuario está integrada. **Vive en:** `apps/web/src/hooks/useWalletBalances.ts` . **Estado:** producción-ready.

Tabla de actividad real Fuji

Lectura del explorador público (Routescan API) de transacciones reales (AVAX + ERC-20) de la wallet conectada. **Por qué importa:** en ausencia de IFC trades reales, mostramos algo verificable y honesto. **Vive en:** `apps/web/src/lib/client/fuji.ts` , `apps/web/src/components/wallet/FujiActivityTable.tsx` . **Estado:** producción-ready.

Loading screen con shader animation

Three.js shader entre login y dashboard (4 segundos). **Por qué importa:** brand polish para captar atención del jurado. **Vive en:** `apps/web/src/components/loading/DashboardLoadingScreen.tsx` . **Estado:** producción-ready.

Connect-wallet gating

Sin wallet conectada: dashboard, órdenes y trades muestran un CTA grande "Conecta tu wallet" en lugar de datos sintéticos. **Por qué importa:** honestidad — un usuario logueado sin wallet no debe ver portfolio fake. **Vive en:** `apps/web/src/components/wallet/ConnectWalletPrompt.tsx` . **Estado:** producción-ready.

3. Tecnologías y librerías — contexto por dependencia

Formato por entrada: **qué es** · **por qué la elegimos** (con alternativa descartada) · **para qué la usamos aquí** · **dónde vive** · **estado**.

Frontend

Next.js 15 (App Router, Turbopack)

Framework React full-stack. Lo elegimos por App Router (server components por default → menos JS al cliente), API Routes integradas (no necesitamos Express aparte) y adopción enterprise. Alternativas descartadas: Remix (menos adopción en fintech), Vite + Express (más boilerplate, sin SSR optimizado). Usamos: todo `apps/web` (frontend + 22 API routes). **Estado:** producción-ready.

React 19

Library UI base. Vino con Next 15. Servidor components reducen el bundle cliente significativamente vs React 18. Usamos: todos los componentes. Estado: producción-ready.

TypeScript estricto (`noUncheckedIndexedAccess`)

Tipado estático. La opción `noUncheckedIndexedAccess` fuerza checks en accesos `arr[i]` (devuelve `T | undefined`), evitando bugs típicos. Alternativas: JS plano (descartado para hackathon serio). Usamos: todo el monorepo. Estado: producción-ready.

Tailwind CSS 3.4

Utility-first CSS. Elegido por velocidad de iteración + tokens consistentes vía `tailwind.config.ts`. Alternativas: CSS Modules (más boilerplate), styled-components (runtime cost). Usamos: todo el styling. Estado: producción-ready. Notable: tokens semánticos (`bg-canvas` , `text-foreground`) abstraen light/dark.

shadcn/ui + Radix UI primitives

Componentes accesibles sin opinión visual fuerte. shadcn nos da el copy-paste de patrones probados, Radix da el comportamiento accesible (focus trapping, keyboard nav). Alternativas: Material-UI (visualmente pesado), Headless UI (menos primitives). Usamos: todos los primitives en `packages/ui`. Estado: producción-ready.

Framer Motion 11

Animaciones declarativas. Elegido por API simple + performance. Alternativas: GSAP (más capaz pero más curva), CSS-only (más fricción para coordinar). Usamos: transiciones de páginas, loading screen, theme toggle morphing. Estado: producción-ready.

Zustand 5

Estado cliente persistido. Elegido por simplicidad (vs Redux + middlewares). Alternativas: Redux Toolkit (overhead innecesario), Jotai (más nuevo, menor tracción). Usamos: `onboardingStore` , `uiStore` (sidebar collapsed, command palette open). Estado: producción-ready.

TanStack Query v5

Data fetching + cache + stale management. Estándar de facto en React. Alternativas: SWR (similar, menos features), fetch crudo (sin cache). Usamos: todas las llamadas a `/api/*` , mocks vía `apiOrMock` . Estado: producción-ready.

TanStack Table v8

Tablas avanzadas headless (sorting, filtering, pagination). Alternativas: AG Grid (overkill), tabla manual (mucho boilerplate). Usamos: DataTable component en `packages/ui`, todas las tablas del admin. Estado: producción-ready.

React Hook Form + Zod resolvers

Forms con validación tipada. Elegido por performance (no re-renders innecesarios) + integración Zod (mismo schema en backend y frontend). Alternativas: Formik (más re-renders), uncontrolled inputs (sin validación). Usamos: login, register, profile, multi-step new offering. Estado: producción-ready.

wagmi v2 + viem v2

Hooks React + cliente Ethereum tipado. wagmi sobre viem es la pareja recomendada por el ecosistema. Alternativas: ethers.js (más viejo, menos tipos), web3.js (legado). Usamos: wallet connect, balance reads, EIP-712 signing. Estado: producción-ready.

RainbowKit 2

UI de conexión de wallet con multi-chain support. Alternativas: ConnectKit (similar, menos branding options), wallet-by-wallet manual (mucho trabajo). Usamos: modal de conexión con Core Wallet primero. Estado: producción-ready.

Recharts 2

Charts React. Elegido por flexibilidad + theming dark/light. Alternativas: Tremor (más opinionado), Chart.js (más imperativo). Usamos: AreaTrend, AllocationDonut, Sparkline. Estado: producción-ready.

date-fns 4

Formateo de fechas con i18n. Elegido por tree-shaking (importas sólo las funciones que usas) y locale es. Alternativas: moment (deprecado), Day.js (más limitado). Usamos: format en tablas de trades/orders/activity. Estado: producción-ready.

lucide-react

Icon set. Elegido por consistencia visual + tree-shaking. Alternativas: Heroicons (también buena), Font Awesome (heavy). Usamos: todos los iconos de la UI. Estado: producción-ready.

cmdk

Command palette (⌘K). Elegido por adopción de patrón Linear/Stripe. Usamos: `CommandPalette` component, montado en topbar. Estado: producción-ready.

class-variance-authority + clsx + tailwind-merge

CVA: variantes de componentes (Button con primary/secondary/ghost). clsx + tailwind-merge: composición segura de classes. Alternativas: classNames (sin merge inteligente). Usamos: todos los componentes con variants. Estado: producción-ready.

Sonner

Toast notifications. Elegido por UX premium (animaciones, swipe to dismiss). Alternativas: react-hot-toast (similar). Usamos: feedback de mutaciones (login, profile update, etc.). Estado: producción-ready.

next-themes

Toggle light/dark con persistencia + SSR-safe. Elegido por integración Next nativa. Alternativas: implementación manual (riesgo de FOUC). Usamos: ThemeProvider + useTheme hook. Estado: producción-ready.

Three.js

WebGL 3D rendering. Usado SOLO para el shader del loading screen post-login. Alternativas: PixiJS (menos adopción), CSS animations (no podían lograr el efecto). Usamos:

`apps/web/src/components/ui/shader-animation.tsx` . Estado: producción-ready, optimizado para perf.

GSAP

Animation library. Usado para los tweens del Hero canvas (rotation, atmosphereShift, glitch). Elegido sobre Framer Motion para esto porque GSAP maneja mejor animaciones continuas/loops que Framer (mejor para canvas). Usamos: `apps/web/src/components/ui/artificial-hero.tsx` . Estado: producción-ready.

Backend

Node.js 20

Runtime JS. Elegido por LTS + soporte ESM nativo. Alternativas: Bun (más rápido pero menos battle-tested), Deno (sin npm ecosystem completo). Usamos: API routes (vía Next), indexer (ESM puro). Estado: producción-ready.

Next.js Route Handlers

Endpoints HTTP. Vienen con Next 15. Beneficio: comparten tipos con el frontend, deploy a Vercel sin servidor aparte. Alternativas: Express/Fastify aparte (más complejidad). Usamos: 22 endpoints en `apps/web/src/app/api/` . Estado: producción-ready.

Prisma ORM 5

ORM con tipado. Elegido por DX (autocomplete + migrations + Studio). Alternativas: Drizzle (más nuevo, menos features), Sequelize (legado). Usamos: 11 modelos + migrations + seed. Estado: producción-ready.

PostgreSQL 16 (Supabase)

Base de datos relacional. Elegido por precisión Decimal (crítico para money), transacciones, ACID.

Alternativas: MongoDB (Decimal precision pobre, fintech standard es SQL), MySQL (menos features).

Usamos: hosted en Supabase free tier. Estado: producción-ready.

Redis 7 (Upstash)

In-memory store + Streams. Elegido por Streams (event bus simple) + Sorted Sets (orderbook cache) + sliding window (rate limit). Alternativas: Kafka (overkill para hackathon), RabbitMQ (más complejo).

Usamos: Streams para eventos del mock chain, BullMQ queues, rate limit. Estado: producción-ready (Upstash hosted).

BullMQ 5

Queues + workers sobre Redis. Elegido por DX (UI Bull Board) + retries automáticos. Alternativas: Agenda (menos mantenido). Usamos: matching processor, notifications processor. Estado: producción-ready (workers no corren persistentes en hackathon).

jose (JWT)

Firma y verificación JWT. Elegido sobre NextAuth porque NextAuth es overkill cuando no necesitas OAuth providers y queremos cookie httpOnly simple. Alternativas: jsonwebtoken (CommonJS, más viejo).

Usamos: sign + verify en login/session. Estado: producción-ready.

bcryptjs

Password hashing. Elegido sobre `bcrypt` nativo porque bcryptjs es JS puro (sin compilación nativa, deploy más simple). Trade-off: ~30% más lento que bcrypt nativo, irrelevante para login throughput.

Usamos: hash al register, verify al login. Estado: producción-ready.

pino + pino-pretty

Logging estructurado. Elegido por performance (más rápido que Winston) + redaction de PII built-in.

Alternativas: Winston (más lento, más viejo). Usamos: todos los API routes loguean con redact paths configurados (RFC, CURP, email parcial). Estado: producción-ready.

ioredis

Cliente Redis TCP. Elegido sobre `redis` package por API más rica (mejor soporte de Streams, Cluster).

Usamos: en backend (rate limit, BullMQ) e indexer (consumer Streams). Estado: producción-ready.

viem (en backend)

Para `recoverMessageAddress` en SIWE-like flow + verificación de firmas EIP-712. Misma library que en frontend, tipos consistentes. Usamos: `linkWallet` service, futura verificación de orders. Estado:

producción-ready.

Zod

Schema validation runtime + tipos derivados. Usamos: cada API route valida body con `Schema.parse`. Schemas viven en `packages/shared/src/dto/` y son los mismos que el frontend importa para react-hook-form. Estado: producción-ready.

Smart contracts (preparado, no deployado)

Solidity 0.8.24 + vialR

Lenguaje + compilador. Elegimos 0.8.24 por overflow checks built-in y custom errors. vialR optimiza más agresivamente. Estado: contratos compilan, no testeados ni deployados.

Hardhat 2.x + hardhat-toolbox

Framework de desarrollo Solidity. Elegido sobre Foundry porque Hardhat tiene mejor integración TS (typechain, scripts en TS). Foundry es más rápido para tests pero el equipo ya sabe Hardhat. Usamos: compile + scripts de deploy. Estado: setup ready.

OpenZeppelin contracts 5

Implementaciones audit-ready de ERC-20, AccessControl, Pausable, etc. Elegido por ser estándar de la industria. Usamos: como base de SecurityToken. Estado: imports listos, no usados extensivamente todavía.

Estándar ERC-3643 (T-REX)

Standard de security tokens regulados. Elegido sobre ERC-1404 (más limitado) y ERC-1400 (menos adopción). Razón: identity + compliance modules separados del token, evolucionables sin redeploy. Usamos: como referencia arquitectónica para nuestros contratos. Estado: arquitectura inspirada, implementación parcial.

Mock blockchain SDK

TypeScript adapters in-memory

Cinco interfaces (Identity, Compliance, SecurityToken, Settlement, Orderbook) con implementaciones mock que mantienen estado en Maps + emiten eventos sintéticos. Por qué: nos permite construir todo lo demás sin esperar contratos. Estado: completo.

EventBus tipado + Redis Streams

EventBus en proceso (typed) + persistencia a Redis Stream `mock-chain-events`. El indexer consume el stream con consumer group. Por qué dual: in-process para tests unitarios, Stream para indexer real.

Estado: completo.

Infraestructura / DevOps

pnpm 10 + workspaces

Package manager + monorepo. Elegido sobre npm/yarn por velocidad (~3x) y manejo estricto de node_modules (no dep ghosting). Usamos: 8 packages en el workspace. Estado: producción-ready.

Turborepo 2

Task pipeline cache. Elegido por integración nativa con pnpm + cache local + remote cache opcional. Alternativas: Nx (más opinado, mayor curva). Usamos: `pnpm dev` levanta web+indexer en paralelo. Estado: producción-ready.

Husky + lint-staged

Git hooks pre-commit. Husky configura el hook, lint-staged corre prettier+eslint sólo en archivos staged. Por qué: commits siempre formateados. Estado: configurado, podría tener más cobertura.

ESLint 9 + Prettier 3

Linting + formatting. Estándar. Usamos: configs propias en `packages/config/`. Estado: producción-ready.

Servicios externos

Supabase (Postgres hosted)

Postgres-as-a-service. Elegido sobre Docker local porque permite a cualquiera del equipo levantar el proyecto sin instalar Docker. Free tier es suficiente para hackathon. Estado: producción-ready, URL en `.env`.

Upstash (Redis Streams hosted)

Redis-as-a-service. Mismo razonamiento que Supabase. Free tier OK. Notable: Upstash soporta Redis Streams (no todos los Redis hosted lo hacen). Estado: producción-ready.

Routescan / Snowtrace API

Lectura de transacciones reales en Fuji. API pública, sin key. Usamos para mostrar actividad real on-chain del wallet del usuario. Estado: producción-ready.

Avalanche Fuji testnet RPC

Endpoint público para leer chain state. Usado por wagmi/viem. Estado: producción-ready.

Pinata (IPFS)

Almacenamiento descentralizado para prospectos de ofertas. Planeado, no implementado. Estado: pendiente.

Reown (WalletConnect Cloud)

Project ID para WalletConnect (mobile QR scan). Opcional — sin él funciona con browser-extension wallets. Estado: pendiente (project ID no configurado).

4. Componentes del sistema — contexto

Smart contracts (13)

Cada uno con responsabilidad única:

- **IdentityRegistry**: mapea wallet a identidad legal verificada + claims (KYC, jurisdicción, acreditación). Sólo wallets registradas pueden holdear/operar SecurityTokens. Quien lo necesita entender: dev backend, auditor, legal.
- **ClaimIssuer**: firma claims (Arkangeles actúa como issuer en MVP). Verifica firmas EIP-712. Auditor debe revisar la verificación.
- **ComplianceRegistry**: orquesta los módulos de compliance por token. Cada SecurityToken puede tener su propia configuración.
- **HoldingPeriodModule**: bloquea transferencias antes del lockup configurado.
- **MaxHoldersModule**: limita número máximo de holders por oferta (regla CNBV).
- **JurisdictionModule**: permite/bloquea por jurisdicción del receptor (lee identity).
- **MaxInvestmentModule**: tope por inversionista no calificado.
- **SecurityToken**: ERC-20 con hooks de compliance + freeze + forcedTransfer + pause. Cada **transfer** consulta IdentityRegistry y todos los módulos de Compliance.
- **TokenFactory**: despliega SecurityToken por cada oferta IFC.
- **OrderBook**: registro on-chain (opcional) de órdenes para auditoría regulatoria.
- **Settlement**: atomic swap token vs USDC con verificación de firmas EIP-712.
- **Escrow**: placeholder para settlement diferido (T+1).
- **MockUSDC**: stablecoin de pruebas.

API endpoints (22)

Agrupados por dominio:

- **Auth**: register, login, logout, session
- **Users**: GET/PATCH /me, POST /me/wallet
- **KYC**: start, webhook

- **Offerings:** list, detail, create, cap-table
- **Orders:** create (con EIP-712), cancel, book by offering, mine
- **Match:** execute (trigger demo)
- **Portfolio:** by wallet
- **Trades:** list con filtros
- **Admin:** freeze, unfreeze, whitelist, audit-log
- **Compliance:** check (para validar transferencias)

Todos validan con Zod, manejan errores via `withErrorHandler` , loggean con pino, aplican RBAC. La lógica vive en services bajo `apps/web/src/lib/server/services/` .

Modelos de base de datos (12)

- `User` (id, email, password_hash, role, firstName, lastName)
- `Identity` (wallet, kycStatus, jurisdiction, accredited, claimHash, frozen)
- `Wallet` (address, isPrimary, linkedAt)
- `Issuer` (name, cnbvLicense, kyclssuerAddress)
- `Offering` (tokenAddress, name, symbol, prospectusIpfs, supply, price, lockup, maxHolders, allowedJurisdictions, status)
- `CapTableEntry` (offeringId, wallet, balance, percentOfTotal)
- `Order` (orderHash, makerWallet, side, qty, price, signature, status)
- `Trade` (buyOrderId, sellOrderId, qty, price, txHash, blockNumber)
- `KycRecord` (provider, payload Json, status, externalId)
- `AuditLog` (action, actor, target, payload, txHash) — append-only
- `Notification` (type, title, body, readAt)
- `ProcessedEvent` (eventId) — idempotencia del indexer

Eventos del mock blockchain

Lista completa: `IdentityRegistered` , `IdentityRemoved` , `WalletFrozen` , `WalletUnfrozen` , `TokenDeployed` , `Transfer` , `ForcedTransfer` , `OrderPosted` , `OrderCancelled` , `OrderFilled` , `TradeExecuted` . Cada uno tiene un handler en el indexer que actualiza Postgres en transacción.

Páginas frontend (~30)

Agrupadas por route group:

- `(landing)` — `/`
- `(auth)` — login, register, forgot-password
- `(onboarding)` — 4 pasos
- `(app)/investor` — dashboard, portfolio, offerings, orderbook detalle, orders, trades, activity, watchlist, profile
- `(app)/issuer` — dashboard, offerings (list + new + detail con cap table, holders, analytics)

- `(app)/admin` — dashboard, investors, compliance, jurisdictions, audit log
-

5. Decisiones arquitectónicas — contexto extendido

Subnet propia en Avalanche vs C-Chain

Decisión: subnet permissioned con validadores de la IFC para producción. **Por qué:** validadores controlados por la IFC + 2-3 partners (no nodos públicos arbitrarios) → CNBV puede auditar quién valida; gas pagado en stablecoin (no AVAX) → UX bancaria sin volatilidad cripto; transacciones permissioned a nivel de protocolo (no sólo a nivel de smart contract) → cumplimiento regulatorio nativo, no opcional.

Trade-off: más complejidad de infra (validadores, AvaCloud), pierdes acceso a tooling C-Chain native (algunos exploradores). **Estado:** planeado para producción. El demo del hackathon corre sobre C-Chain Fuji por costo y simplicidad.

Matching off-chain + settlement on-chain

Decisión: orderbook y matching engine en backend (Postgres + Redis); settlement on-chain ejecuta atómicamente. **Por qué:** orderbook 100% on-chain es caro (gas por orden + cancelación) y lento (espera por bloque). Matching off-chain con firmas EIP-712 + ejecución atómica on-chain da UX de exchange tradicional con garantías de blockchain (settlement no se puede falsificar, compliance se aplica on-chain). Es el patrón de 0x, dYdX v3. **Trade-off:** dependes de un matcher centralizado (riesgo de censura).

Mitigación: orden firmada off-chain es portable, usuarios pueden ir a otro matcher. **Estado:** diseño aceptado, matcher mock corriendo, settlement contract escrito.

ERC-3643 (T-REX) en lugar de ERC-1404 / ERC-1400

Decisión: base ERC-3643 con identity + compliance modules separados. **Por qué:** ERC-3643 es el estándar de facto para security tokens regulados (Tokeny, varias instituciones europeas). Identity Registry + módulos de compliance evolucionables sin redeploy del token. ERC-1404 más simple pero sin identity registry. ERC-1400 más viejo, menos adopción. **Trade-off:** más contratos (mayor superficie de auditoría), curva de aprendizaje mayor. **Estado:** arquitectura inspirada, implementación parcial.

PostgreSQL en lugar de MongoDB

Decisión: Postgres. **Por qué:** precisión Decimal nativa (crítica para money — Mongo no la soporta bien en Prisma), transactions ACID, fintech standard. Bancos y fintech serias usan SQL. **Trade-off:** schema más rígido (necesitas migrations vs schemas flexibles). **Estado:** decisión validada, migrations corriendo.

Mock blockchain layer con misma interfaz que el real

Decisión: SDK con interfaces tipadas, dos implementaciones (mock + futuro real). **Por qué:** permite construir frontend, backend, indexer, KYC sin esperar contratos deployados; swap de mock a real es sólo cambiar `mode: 'mock'` por `mode: 'avalanche'`. Tests unitarios usan el mock. **Trade-off:** mantener el contrato interfaz al día requiere disciplina. **Estado:** mock completo y funcional.

jose para JWT en lugar de NextAuth

Decisión: jose + cookie httpOnly manual. **Por qué:** NextAuth es overkill cuando no necesitas OAuth providers (no usamos Google/GitHub login). jose es 8KB vs NextAuth ~200KB+. Cookie httpOnly + JWT firmado es estándar y simple. **Trade-off:** tienes que escribir tu propio middleware de auth (ya lo hicimos). **Estado:** producción-ready.

Cookie httpOnly para sesión

Decisión: cookie httpOnly + Secure + SameSite=Lax. **Por qué:** localStorage es vulnerable a XSS (JS puede leerlo). Cookie httpOnly no es accesible desde JS. **Trade-off:** requiere CSRF protection (mitigado por SameSite). **Estado:** producción-ready.

Connect-wallet gating del dashboard

Decisión: sin wallet conectada, dashboard/orders/trades muestran un CTA grande en lugar de datos sintéticos. **Por qué:** honestidad. Usuario logueado pero sin wallet no debe ver portfolio inventado; eso degrada confianza. **Trade-off:** primer impacto visual menos "wow" si no conecta. **Estado:** implementado.

Trades/órdenes IFC en empty state honesto + actividad real Fuji

Decisión: tabla "Trades IFC" muestra empty state con disclaimer ("aparecerán cuando los contratos se deployen") + sección "Actividad on-chain en Fuji" con TUS transacciones reales del wallet vía Routescan. **Por qué:** en lugar de mostrar trades fake derivados de wallet, ser explícito sobre qué es real y qué no. **Estado:** implementado.

pnpm workspaces (no npm/yarn)

Decisión: pnpm. **Por qué:** ~3x más rápido en install, dep ghosting estricto (no puedes importar deps no declaradas), workspaces robustos. **Trade-off:** algunos packages no funcionan con strict mode (ej. `@prisma/client` requirió ajuste). Documentado y resuelto. **Estado:** producción-ready.

Supabase + Upstash hosted vs Docker local

Decisión: hosted free tier. **Por qué:** cualquier persona del equipo levanta el proyecto sin instalar Docker (instalar Docker en Windows lleva ~30 min y rompe a algunos). Hosted es 5 min de setup. **Trade-off:** depende de internet, latencia para queries. **Estado:** producción-ready, alternativa Docker disponible vía `docker-compose.yml`.

Redis Streams para event bus

Decisión: Redis Streams + consumer groups. **Por qué:** liviano, viene gratis con Redis (ya teníamos para BullMQ), perfect at-least-once semantics. Para hackathon scope, Kafka/SNS son overkill. **Trade-off:** no escala a millones de eventos/seg como Kafka. Suficiente para 100x el volumen target. **Estado:** producción-ready.

Core Wallet priorizada en grupo Avalanche

Decisión: en el modal de RainbowKit, Core (wallet oficial de Avalanche) está en su propio grupo arriba de "Más opciones". **Por qué:** alineación de marca con Avalanche + es la wallet con mejor soporte para subnets. **Estado:** implementado.

6. Flujos clave — contexto

Onboarding de inversionista

1. Usuario registra (email, password, nombre, apellido). Backend crea User en Postgres.
2. Redirige a `/onboarding`. Wizard de 4 pasos: datos personales adicionales → KYC mock (subir INE, comprobante) → connect wallet (RainbowKit) → SIWE-like signature → backend verifica firma + asocia wallet a User + crea registro Identity.
3. Backend (en producción real) llama a `IdentityRegistry.registerIdentity(wallet, claim)` on-chain.
4. Wallet queda elegible para holdear y operar SecurityTokens.

Componentes involucrados: register page, onboarding pages, `/api/auth/register`, `/api/kyc/start`, `/api/users/me/wallet`, `userRepo`, `IdentityRegistry` (futuro).

Emisión de oferta

1. Operador IFC (rol issuer) entra a `/issuer/offerings/new`. Multi-step form.
2. Sube prospecto (planeado: a IPFS, hash queda en Postgres).
3. Configura supply, lockup, max holders, jurisdicciones permitidas, requisito de acreditación.
4. Backend valida + persiste como `Offering` con status `draft`.
5. (Futuro) Backend llama a `TokenFactory.deployToken(...)` que despliega un nuevo SecurityToken con configuración. Token address se guarda en `Offering.tokenAddress`.
6. Mint inicial a wallets de inversionistas primarios.

Trade en mercado secundario (el demo principal)

1. Inversionista A entra a un offering, ve orderbook, decide vender. Click "Place Order" → form.
2. Frontend construye el `OrderPayload` (offeringId, side=sell, qty, price, expiresAt, salt) y pide a la wallet firmar EIP-712 sobre el dominio del Settlement contract.
3. POST `/api/orders` con la firma. Backend verifica la firma con viem (re-derive hash, recover signer = maker). Persiste Order en Postgres + push a Redis Sorted Set del orderbook.
4. Inversionista B ve la orden en el orderbook. Firma orden de compra que cruza.
5. Matching engine (servicio backend) encuentra el match. Construye una transacción de Settlement.
6. Settlement.executeMatch on-chain (atómico):
 - Verifica firmas
 - SecurityToken consulta ComplianceRegistry: ¿B puede recibir?

- Si OK: token transfer A→B + USDC transfer B→A + fee a feeRecipient
- Emite evento `TradeExecuted`

7. Indexer escucha evento + inserta `Trade` en Postgres + actualiza `cap_table`.

8. Frontend (TanStack Query) refetch → ambos ven el cambio.

Componentes: orderbook UI, OrderForm, `/api/orders`, `orderService`, `matchingService`, `Settlement` contract, indexer handlers.

Compliance ops (freeze / forced transfer)

1. Compliance officer entra a `/admin/investors`. Tabla con todos los users.
2. Click en una fila → drawer con detalle. Botón "Freeze wallet" → confirma.
3. POST `/api/admin/freeze`. Backend: (a) escribe AuditLog ANTES de cualquier cosa, (b) llama a `SecurityToken.freezeWallet(wallet, true)` on-chain (futuro).
4. Indexer recibe `WalletFrozen` event → marca `Identity.frozen = true`.
5. Próxima orden de esa wallet rechaza con compliance error.

Distribución de dividendos

Pendiente. Diseño: contrato `Dividends.sol` que recibe USDC, calcula pro-rata por holder en cap table snapshot, holders pueden `claim()`. No implementado en este hackathon.

7. Compliance / Regulatorio — contexto extendido

Marco legal aplicable

- **Ley para Regular las Instituciones de Tecnología Financiera** (Ley Fintech, 2018): regula a las IFCs como categoría. Establece límites por inversionista por oferta y total anual.
- **Disposiciones de Carácter General aplicables a las IFC** (CNBV): operativos detallados — KYC, audit log, reportes mensuales.
- **Ley del Mercado de Valores (LMV)**: relevante porque las participaciones IFC NO son securities bajo LMV (matiz importante; si lo fueran, requerirían registro como emisora bursátil).
- **LFPDPPP**: protección de datos personales — relevante para el manejo de KYC, RFC, CURP.

Conceptos clave

- **Inversionista calificado**: persona física que cumple ingresos/patrimonio/experiencia exigidos por CNBV. Puede invertir sin topes.
- **Inversionista no calificado**: tope por oferta (~7,500 UDIs ≈ 60k MXN) y total anual.
- **Jurisdicción permitida**: ISO 3166-1 numeric (484 = MX). Por default permitimos sólo MX; expansión a otros países requiere registro adicional.

- **Holding period / lockup:** período mínimo durante el cual no se puede vender (típicamente 12 meses en primaria).
- **Max holders por oferta:** límite de inversionistas concurrentes (regla CNBV implícita).
- **Audit log immutable:** requisito CNBV; cumplimos con tabla append-only + eventos on-chain.
- **Forced transfer:** capacidad de mover tokens sin consentimiento del holder (recovery por pérdida de claves o orden judicial). Implementado como `onlyAgent`.
- **Freeze wallet:** bloquea transferencias salientes (orden judicial / sospecha AML).
- **KYC issuer:** entidad autorizada para emitir claims firmadas (Arkangeles en MVP).
- **Sandbox CNBV:** figura legal para operar modelos novedosos sin licencia plena. Ruta de adopción planeada post-hackathon.

Cómo cumplimos cada requisito (mapeo)

- Validación de jurisdicción → `JurisdictionModule.canTransfer` lee identity del receptor.
- Tope inversionista no calificado → `MaxInvestmentModule.canTransfer` lee accredited flag.
- Lockup → `HoldingPeriodModule.canTransfer` chequea block.timestamp vs lockupUntil.
- Límite holders → `MaxHoldersModule` incrementa cuando hay nuevo holder.
- Forced transfer / freeze → `SecurityToken.forcedTransfer` y `freezeWallet`, `onlyAgent`.
- Audit log → tabla `audit_log` + eventos on-chain inmutables.
- KYC trazable → `ClaimIssuer` firma claims, `IdentityRegistry` mapea.

8. Seguridad — contexto

- **bcrypt cost factor 10:** balance entre seguridad y latencia de login (~80ms).
- **JWT HS256 con jose:** secret en env, cookie httpOnly + Secure + SameSite=Lax.
- **Rate limiting Redis sliding window:** login 5/min/IP, orders 30/min/wallet, kyc 3/h/user.
- **Validación Zod en boundary:** cada API route valida body antes de tocar lógica.
- **Verificación de firma EIP-712:** en order creation antes de aceptar.
- **SIWE-like flow:** link wallet requiere firma sobre mensaje con dominio + nonce + address.
- **PII redaction en logs:** pino con redact paths para `email`, `password`, `rfc`, `curp`, `name`.
- **RBAC:** middleware `withRole(['admin'])` rechaza con 403 si rol no autorizado.
- **AppError tipado:** códigos estables (`AUTH_INVALID`, `VALIDATION_ERROR`, `RATE_LIMIT`), no leak de stack traces.
- **Idempotencia en webhooks:** KYC webhook usa idempotency key (HMAC del payload).
- **Audit log antes de admin action:** si la acción on-chain falla, el intent queda registrado.
- `noUncheckedIndexedAccess`: TS strict que evita bugs de `arr[i]` undefined.
- **KYC issuer private key en env:** nunca en código, nunca en logs.

9. Estado actual del proyecto

Producción-ready (funciona end-to-end ahora)

- Auth real (login, register, logout, session, profile editor)
- DB schema migrada en Supabase + seed con datos demo
- 22 endpoints API con auth + RBAC + rate limiting + validación
- Mock blockchain SDK con 5 adapters + EventBus + Redis Streams
- Indexer event-driven funcional
- Wallet integration (Core, MetaMask, Coinbase, Rabby, Injected; WalletConnect opcional)
- Dashboard gateado por wallet conectada
- Balances reales AVAX/USDC desde RPC de Fuji
- Tabla actividad real Fuji vía Routerscan
- Loading screen con shader Three.js
- Hero canvas con cosmic animation (azul Arkangeles)
- Architecture viz orbital interactivo
- Light/dark theme toggle (persiste)
- Liquid Glass buttons en CTAs del Hero
- ~30 páginas frontend completas

Mock o placeholder (interfaz lista, sin lógica real on-chain)

- Holdings IFC (no son tokens reales hasta deploy de contratos)
- Mis órdenes IFC (empty state honesto)
- Mis trades IFC (empty state honesto + actividad real Fuji)
- Orderbook animado (datos sintéticos por timer)
- KYC verification (UI mock, sin Truora/Mati real)
- Distribución de dividendos
- Mapa de jurisdicciones (SVG simple, no topología real)

Pendiente

- Deploy real de smart contracts a Fuji
- Tests unitarios de smart contracts
- Auditoría de smart contracts (post-hackathon)
- Integración con KYC provider real
- Workers BullMQ corriendo persistentes
- WalletConnect mobile QR (requiere Reown project ID)
- Pitch deck final
- Video demo
- Subnet propia en AvaCloud (post-hackathon)

- Sandbox CNBV (post-hackathon)

10. Plantilla para el documento final

Para cada ítem que documentes, sigue esta plantilla. Las respuestas viables están en este dossier — sólo reformula:

```
### <Nombre del ítem>

**Qué es:** <una frase técnica específica al proyecto>

**Por qué se eligió / por qué es importante:**
<2-3 líneas con la razón concreta de elegirlo, mencionando la
alternativa que se descartó y por qué>

**Para qué sirve dentro del producto:**
<dónde se usa y qué hace en el flujo del producto. Ejemplo:
"el orderbook lo consume el Settlement contract para encontrar
matches; el frontend lo renderiza animado en /investor/offerings/[id]">

**Estado:** producción-ready · mock · pendiente

**Dónde vive en el repo:**
<ruta o rutas: `apps/web/src/app/api/orders/route.ts`,
`packages/sdk/src/blockchain/mock/`, etc.>

**Quién lo necesita entender:**
<dev backend / dev frontend / auditor de seguridad / equipo legal
de la IFC / jurado / inversionista>
```

Ejemplo aplicado (referencia)

```
### Prisma ORM 5

**Qué es:** ORM TypeScript con migrations declarativas, autocomplete
y Studio (UI para inspeccionar la DB).

**Por qué se eligió / por qué es importante:**
Necesitábamos tipado fuerte entre schema y código (evitar bugs por
tipos en field names) y migrations versionadas. Descartamos Drizzle
porque tenía menos features para nuestro scope, y Sequelize por ser
legado sin soporte TS de primera clase.

**Para qué sirve dentro del producto:**
Define los 12 modelos del dominio (User, Identity, Offering, Order,
Trade, AuditLog, etc.) y genera el client tipado que todos los
servicios del backend usan para queries.

**Estado:** producción-ready

**Dónde vive en el repo:**
`packages/database/prisma/schema.prisma`, `packages/database/src/`,
y consumido en `apps/web/src/lib/server/repositories/*.ts` y
`apps/indexer/src/handlers/*.ts`.

**Quién lo necesita entender:**
Dev backend (todos los días), auditor (esquema de datos = superficie
de ataque), equipo legal (la tabla audit_log es el soporte de
cumplimiento CNBV).
```

11. Glosario rápido (jerga del proyecto)

- **IFC:** Institución de Financiamiento Colectivo (equity crowdfunding regulado por CNBV).
- **CNBV:** Comisión Nacional Bancaria y de Valores. Regulador financiero mexicano.
- **CNBV / Ley Fintech:** marco regulatorio aplicable a las IFCs.
- **Cap table:** tabla de capitalización (quién posee qué porcentaje del equity).
- **EIP-712:** estándar Ethereum para firmar mensajes estructurados (no opaque hashes).
- **ERC-3643 / T-REX:** estándar de security tokens regulados con identity + compliance.
- **ERC-20:** estándar fungible básico (lo que usan stablecoins, governance tokens).
- **Settlement atómico:** ambas partes del swap se ejecutan o ninguna (no hay punto donde sólo una recibió).
- **Lockup:** período obligatorio sin venta tras adquirir.
- **KYC (Know Your Customer):** verificación de identidad regulatoria.
- **AML (Anti-Money Laundering):** prevención de lavado.
- **SIWE (Sign-In With Ethereum):** estándar para autenticar con wallet firmando un mensaje.
- **Avalanche Fuji:** testnet de Avalanche (chainId 43113).

- **Avalanche Subnet:** blockchain custom corriendo sobre Avalanche con validadores propios.
 - **Snowtrace:** explorer público de Avalanche.
 - **wagmi / viem:** stack de librerías React + TS para Ethereum.
 - **RainbowKit:** UI estandarizada de conexión de wallet.
 - **Indexer:** proceso que escucha eventos on-chain y los persiste en BD off-chain queryable.
 - **Mock blockchain layer:** implementación in-memory que se comporta como el chain real (mismo SDK), permitiendo desarrollar sin contratos deployados.
 - **RBAC (Role-Based Access Control):** permisos basados en rol (investor / issuer / admin).
 - **Audit log:** registro inmutable de operaciones (requisito CNBV).
 - **Forced transfer:** transferencia forzada por compliance officer (recovery).
 - **Freeze wallet:** bloqueo de transferencias salientes (orden judicial / AML).
-

Cómo usar este dossier: la persona que documente abre este PDF en un panel y el documento final en otro. Para cada sección del brief, busca aquí el contexto, reformúlalo en la plantilla del punto 10, y verifica que las seis preguntas queden respondidas. El dossier es la fuente; el documento final es la destilación.